MANUAL PASO A PASO CON TDD

LOTERÍA DOLOREÑA

Un juego construido 100% guiado por pruebas, desde cero

Desarrollo Móvil Integral — Proyecto complementario de práctica TDD

16 cartas temáticas de la Independencia y Dolores Hidalgo

1. ¿Por qué este proyecto es perfecto para practicar TDD?

La Lotería Mexicana es el juego de mesa más querido del país: se canta una carta a la vez y cada jugador marca su tabla, buscando completar una línea, una columna, una diagonal o la tabla llena. Detrás de esa simplicidad hay algoritmos genuinos y muy divertidos de construir con TDD: barajar sin perder ni repetir cartas, y detectar patrones ganadores en una cuadrícula.

A diferencia de los proyectos anteriores, en este manual construirás TODA la lógica del juego desde una carpeta vacía, en 10 rondas, sin escribir una sola línea que no haya sido pedida primero por una prueba. La interfaz gráfica llega hasta el final, como una capa delgada sobre una lógica ya probada y confiable.

Las cartas de esta versión

En lugar de El Diablito o La Dama, esta Lotería usa 16 cartas con temática de Dolores Hidalgo y la Independencia: El Cura, La Campana, El Grito, La Parroquia, El Sarape, La Nieve, El Sombrero, La Guitarra, El Caballo, La Bandera, El Insurgente, La Plaza, El Fuego, La Estrella, El Águila y El Tambor.

Reglas simplificadas de esta versión digital

- La tabla es una cuadrícula de 4x4 que contiene las 16 cartas, en orden aleatorio.
- El juego 'canta' cartas una por una, tomadas de una baraja barajada de esas mismas 16 cartas.
- Cuando una carta cantada aparece en la tabla, se marca automáticamente esa celda.
- Se gana ('¡Lotería!') al completar cualquier fila, columna o diagonal completa.

2. Estructura del proyecto

```
lib/
  modelos/
    carta.dart
  logica/
    baraja.dart
    tabla.dart
    verificador_patrones.dart
    motor_juego.dart
  main.dart                    (interfaz grafica, al final)
test/
  baraja_test.dart
  tabla_test.dart
  verificador_patrones_test.dart
  motor_juego_test.dart

flutter create loteria_dolorena
cd loteria_dolorena
```

> ☑ **BUENA PRÁCTICA —** Fíjate que lib/logica/ y lib/modelos/ no tienen ninguna razón para importar Flutter. Toda esta parte del proyecto es Dart puro — eso es lo que la hace tan rápida y agradable de probar.

3. El modelo base: Carta

Antes de la primera prueba, necesitamos un tipo de dato mínimo para representar una carta. Esto no requiere TDD por sí mismo — es solo la estructura de datos sobre la que construiremos toda la lógica.

lib/modelos/carta.dart

```
class Carta {
  final int id;
  final String nombre;
  const Carta({required this.id, required this.nombre});

  @override
  bool operator ==(Object other) => other is Carta && other.id == id;

  @override
  int get hashCode => id.hashCode;

  @override
  String toString() => nombre;
}
```

Sobrescribimos == y hashCode a propósito: dos Cartas con el mismo id se consideran la misma carta, sin importar si son objetos distintos en memoria. Esto será clave más adelante para poder comparar cartas 'cantadas' contra cartas 'en la tabla'.

RONDA 1 — La baraja completa

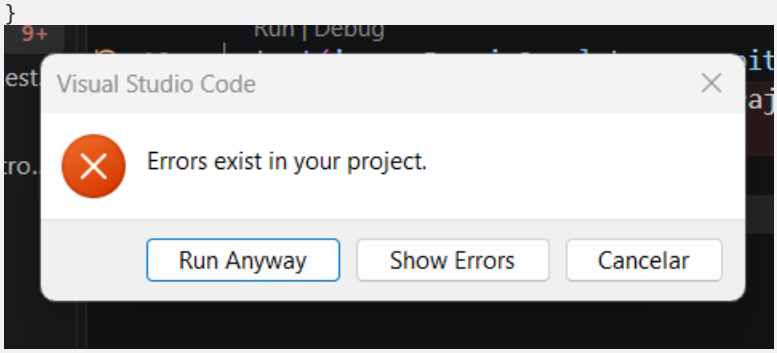
ROJO — Escribe una prueba que falle

test/baraja_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:loteria_dolorena/logica/baraja.dart';

void main() {
  test('crearBarajaCompleta regresa exactamente 16 cartas', () {
    final baraja = Baraja.crearBarajaCompleta();
    expect(baraja.cartasRestantes.length, 16);
  });

  test('crearBarajaCompleta no repite ninguna carta', () {
    final baraja = Baraja.crearBarajaCompleta();
    final idsUnicos = baraja.cartasRestantes.map((c) => c.id).toSet();
    expect(idsUnicos.length, 16);
  });
}
```



VERDE — El mínimo código para pasar

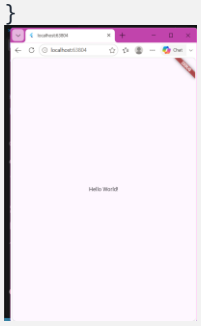
lib/logica/baraja.dart

```
import '../modelos/carta.dart';

class Baraja {
  final List<Carta> cartasRestantes;
  Baraja._(this.cartasRestantes);

  static const List<String> _nombres = [
    'El Cura', 'La Campana', 'El Grito', 'La Parroquia',
    'El Sarape', 'La Nieve', 'El Sombrero', 'La Guitarra',
    'El Caballo', 'La Bandera', 'El Insurgente', 'La Plaza',
    'El Fuego', 'La Estrella', 'El Aguila', 'El Tambor',
  ];

  static Baraja crearBarajaCompleta() {
    final cartas = <Carta>[];
    for (var i = 0; i < _nombres.length; i++) {
      cartas.add(Carta(id: i, nombre: _nombres[i]));
    }
    return Baraja._(cartas);
  }
}
```

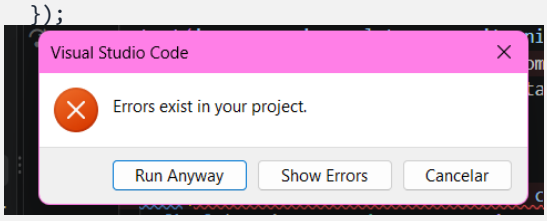


RONDA 2 — Barajar sin perder ni duplicar cartas

ROJO — Escribe una prueba que falle

test/baraja_test.dart (prueba agregada)

```
  test('mezclar conserva las mismas 16 cartas, solo cambia el orden', () {
    final baraja = Baraja.crearBarajaCompleta();
    final idsAntes = baraja.cartasRestantes.map((c) => c.id).toSet();
    baraja.mezclar();
    final idsDespues = baraja.cartasRestantes.map((c) => c.id).toSet();
    expect(idsDespues, idsAntes);
  });
```



VERDE — El mínimo código para pasar

lib/logica/baraja.dart (método agregado)

```
  void mezclar() => cartasRestantes.shuffle();
```

La prueba compara CONJUNTOS de ids (Set), no listas — así no importa el nuevo orden, solo que sigan siendo las mismas 16 cartas.

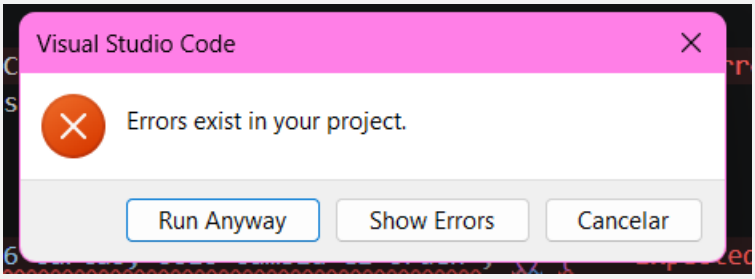
RONDA 3 — Cantar cartas, una por una

ROJO — Escribe una prueba que falle

test/baraja_test.dart (pruebas agregadas)

```
  test('cantarSiguiente entrega una carta y la retira del mazo', () {
    final baraja = Baraja.crearBarajaCompleta();
    final total = baraja.cartasRestantes.length;
    final primera = baraja.cantarSiguiente();
    expect(primera, isNotNull);
    expect(baraja.cartasRestantes.length, total - 1);
  });

  test('cantarSiguiente regresa null cuando ya no hay cartas', () {
    final baraja = Baraja.crearBarajaCompleta();
    for (var i = 0; i < 16; i++) {
      baraja.cantarSiguiente();
    }
    expect(baraja.cantarSiguiente(), isNull);
  });
```



VERDE — El mínimo código para pasar

lib/logica/baraja.dart (método agregado)

```
  Carta? cantarSiguiente() {
    if (cartasRestantes.isEmpty) return null;
    return cartasRestantes.removeAt(0);
  }
```

REFACTOR — Mejora el código sin romper nada

removeAt(0) desplaza todos los elementos restantes de la lista (más lento). Como la baraja ya fue mezclada, el orden no importa — podemos quitar del final con removeLast(), mucho más eficiente. Las pruebas siguen pasando sin cambiar ni una línea de ellas: eso confirma que el refactor no alteró el comportamiento observable.

lib/logica/baraja.dart (refactorizado)

```
  Carta? cantarSiguiente() {
    if (cartasRestantes.isEmpty) return null;
    return cartasRestantes.removeLast();
  }
```

RONDA 4 — La tabla: una cuadrícula de 4x4

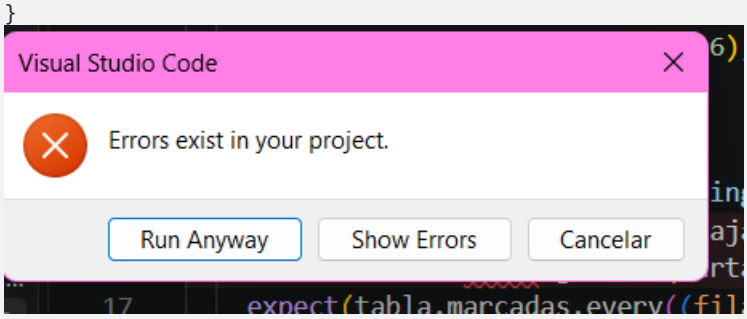
🔴 ROJO — Escribe una prueba que falle

test/tabla_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:loteria_dolorena/logica/baraja.dart';
import 'package:loteria_dolorena/logica/tabla.dart';

void main() {
  test('una tabla nueva contiene las 16 cartas, sin repetir', () {
    final cartas = Baraja.crearBarajaCompleta().cartasRestantes;
    final tabla = Tabla.generar(cartas);
    final todas = tabla.filas.expand((fila) => fila).toList();
    expect(todas.length, 16);
    expect(todas.toSet().length, 16);
  });

  test('una tabla nueva no tiene ninguna celda marcada', () {
    final cartas = Baraja.crearBarajaCompleta().cartasRestantes;
    final tabla = Tabla.generar(cartas);
    expect(tabla.marcadas.every((fila) => fila.every((m) => m == false)), true);
  });
}
```



🟢 VERDE — El mínimo código para pasar

lib/logica/tabla.dart

```
import '../modelos/carta.dart';

class Tabla {
  final List<List<Carta>> filas;
  final List<List<bool>> marcadas;

  Tabla._(this.filas, this.marcadas);

  static Tabla generar(List<Carta> cartas) {
    final copia = List<Carta>.from(cartas)..shuffle();
    final filas = <List<Carta>>[];
    for (var f = 0; f < 4; f++) {
      filas.add(copia.sublist(f * 4, f * 4 + 4));
    }
    final marcadas = List.generate(4, (_) => List.generate(4, (_) => false));
    return Tabla._(filas, marcadas);
  }
}
```

RONDA 5 — Marcar una carta en la tabla

ROJO — Escribe una prueba que falle

test/tabla_test.dart (pruebas agregadas)

```
  test('marcar encuentra la carta en la tabla y marca su celda', () {
    final cartas = Baraja.crearBarajaCompleta().cartasRestantes;
    final tabla = Tabla.generar(cartas);
    final cartaEnTabla = tabla.filas[0][0];

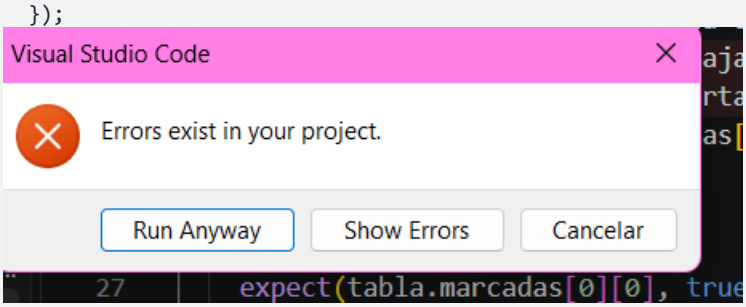
    tabla.marcar(cartaEnTabla);

    expect(tabla.marcadas[0][0], true);
  });

  test('marcar una carta que no esta en la tabla no cambia nada', () {
    final cartas = Baraja.crearBarajaCompleta().cartasRestantes;
    final tabla = Tabla.generar(cartas);
    const cartaAjena = Carta(id: 999, nombre: 'Carta inexistente');

    tabla.marcar(cartaAjena);

    expect(tabla.marcadas.every((fila) => fila.every((m) => m == false)), true);
  });
```



VERDE — El mínimo código para pasar

lib/logica/tabla.dart (método agregado)

```
  void marcar(Carta carta) {
    for (var f = 0; f < filas.length; f++) {
      for (var c = 0; c < filas[f].length; c++) {
        if (filas[f][c] == carta) {
          marcadas[f][c] = true;
          return;
        }
      }
    }
  }
```

Esta prueba solo funciona porque en el modelo Carta sobrescribimos == para comparar por id — de lo contrario, filas[f][c] == carta jamás sería verdadero entre dos objetos distintos.

RONDA 6 — Detectar una línea horizontal ganadora

🔴 ROJO — Escribe una prueba que falle

test/verificador_patrones_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:loteria_dolorena/logica/verificador_patrones.dart';

void main() {
  test('detecta una linea horizontal completa', () {
    final marcadas = [
      [true, true, true, true],
      [false, false, false, false],
      [false, false, false, false],
      [false, false, false, false],
    ];
    expect(VerificadorPatrones.hayLinea(marcadas), true);
  });

  test('no detecta linea si ninguna fila esta completa', () {
    final marcadas = [
      [true, true, true, false],
      [false, false, false, false],
      [false, false, false, false],
      [false, false, false, false],
    ];
    expect(VerificadorPatrones.hayLinea(marcadas), false);
  });
}
```

🟢 VERDE — El mínimo código para pasar

lib/logica/verificador_patrones.dart

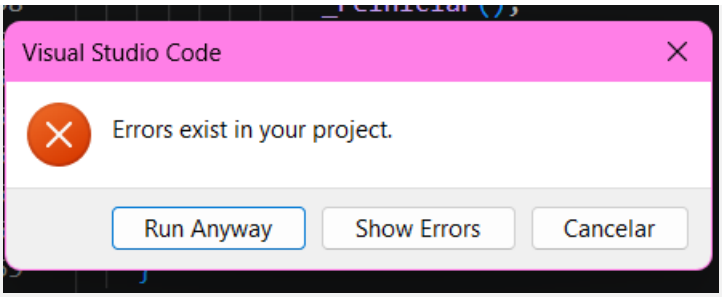
```
class VerificadorPatrones {
  static bool hayLinea(List<List<bool>> marcadas) {
    for (final fila in marcadas) {
      if (fila.every((m) => m)) return true;
    }
    return false;
  }
}
```

RONDA 7 — Detectar una columna ganadora

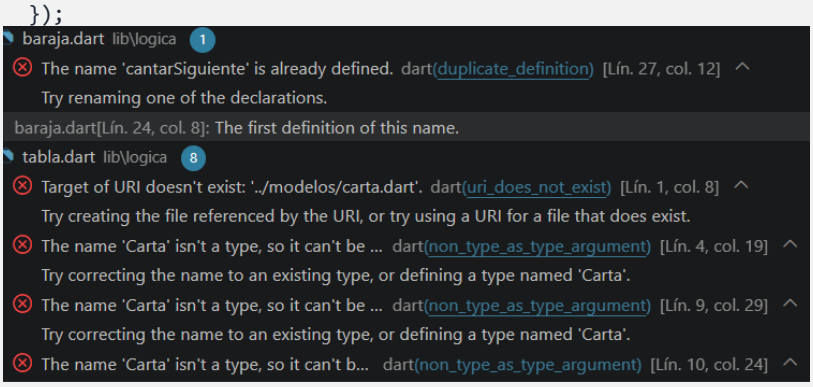
🔴 ROJO — Escribe una prueba que falle

test/verificador_patrones_test.dart (pruebas agregadas)

```
  test('detecta una columna vertical completa', () {
    final marcadas = [
      [true, false, false, false],
      [true, false, false, false],
      [true, false, false, false],
      [true, false, false, false],
    ];
    expect(VerificadorPatrones.hayColumna(marcadas), true);
  });
```




```
  test('no detecta columna si ninguna esta completa', () {
    final marcadas = [
      [true, false, false, false],
      [true, false, false, false],
      [false, false, false, false],
      [true, false, false, false],
    ];
    expect(VerificadorPatrones.hayColumna(marcadas), false);
  });
```

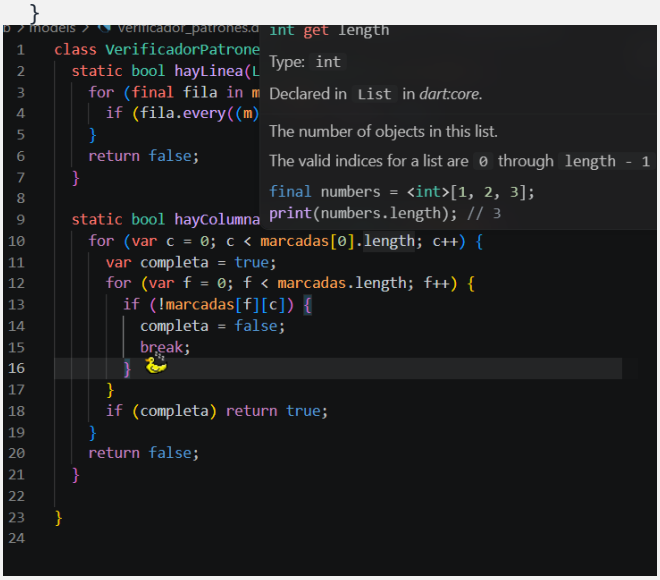


VERDE — El mínimo código para pasar

lib/logica/verificador_patrones.dart (método agregado)

```
  static bool hayColumna(List<List<bool>> marcadas) {
    for (var c = 0; c < marcadas[0].length; c++) {
      var completa = true;
      for (var f = 0; f < marcadas.length; f++) {
        if (!marcadas[f][c]) {
          completa = false;
          break;
        }
      }
      if (completa) return true;
    }
    return false;
```

```
  }
```



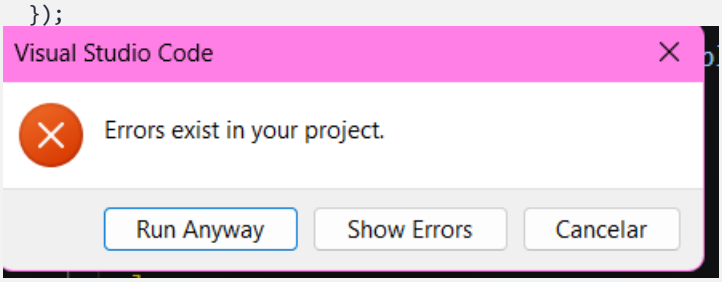
RONDA 8 — Detectar una diagonal ganadora

🔴 ROJO — Escribe una prueba que falle

test/verificador_patrones_test.dart (pruebas agregadas)

```
  test('detecta la diagonal principal completa', () {
    final marcadas = [
      [true, false, false, false],
      [false, true, false, false],
      [false, false, true, false],
      [false, false, false, true],
    ];
    expect(VerificadorPatrones.hayDiagonal(marcadas), true);
  });

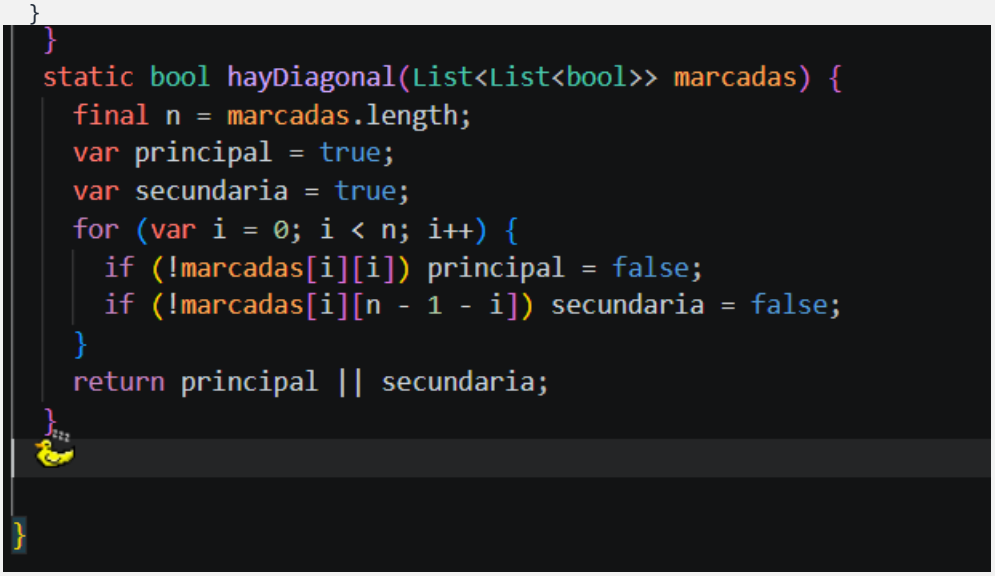
  test('detecta la diagonal secundaria completa', () {
    final marcadas = [
      [false, false, false, true],
      [false, false, true, false],
      [false, true, false, false],
      [true, false, false, false],
    ];
    expect(VerificadorPatrones.hayDiagonal(marcadas), true);
  });
```



🟢 VERDE — El mínimo código para pasar

lib/logica/verificador_patrones.dart (método agregado)

```
  static bool hayDiagonal(List<List<bool>> marcadas) {
    final n = marcadas.length;
    var principal = true;
    var secundaria = true;
    for (var i = 0; i < n; i++) {
      if (!marcadas[i][i]) principal = false;
      if (!marcadas[i][n - 1 - i]) secundaria = false;
    }
    return principal || secundaria;
  }
```



REFACTOR — Mejora el código sin romper nada

Las tres funciones (hayLinea, hayColumna, hayDiagonal) siempre se usarán juntas para decidir si hay '¡Lotería!'. Agregamos un método combinado, sin borrar los anteriores (siguen siendo útiles por separado y sus pruebas los siguen validando).

lib/logica/verificador_patrones.dart (método combinado agregado)

```
  static bool hayGanador(List<List<bool>> marcadas) {
    return hayLinea(marcadas) || hayColumna(marcadas) || hayDiagonal(marcadas);
  }
```

RONDA 9 — Preparar el terreno para probar el juego completo

Antes de integrar todo en un MotorJuego, notamos un problema: para probar 'se cantan 4 cartas y se gana', necesitamos control total sobre el orden de la baraja y el acomodo de la tabla — y ahora mismo ambas siempre mezclan al azar. Esta ronda no agrega una funcionalidad nueva del juego; mejora el diseño para que sea posible probarlo.

🔴 ROJO — Escribe una prueba que falle

La 'prueba que falla' aquí es conceptual: si intentamos escribir la prueba de integración de la Ronda 10 con el diseño actual, no podríamos controlar el orden — la prueba sería aleatoria e inútil.

🟢 VERDE — El mínimo código para pasar

Agregamos constructores con nombre que reciben el orden exacto de cartas, sin mezclar. Las clases siguen funcionando igual para el juego real (que sí quiere aleatoriedad), pero ahora también son controlables desde una prueba.

lib/logica/baraja.dart (constructor agregado)

```
  Baraja.desdeCartas(List<Carta> cartas) : cartasRestantes = List<Carta>.from(cartas);

  static Baraja crearBarajaCompleta() {
    final cartas = <Carta>[];
    for (var i = 0; i < _nombres.length; i++) {
      cartas.add(Carta(id: i, nombre: _nombres[i]));
    }
    return Baraja.desdeCartas(cartas);
  }
```

lib/logica/tabla.dart (constructor agregado)

```
  Tabla.desdeFilas(this.filas) : marcadas = List.generate(4, (_) => List.generate(4, (_) =>
false));

  static Tabla generar(List<Carta> cartas) {
    final copia = List<Carta>.from(cartas)..shuffle();
    final filas = <List<Carta>>[];
    for (var f = 0; f < 4; f++) {
      filas.add(copia.sublist(f * 4, f * 4 + 4));
    }
    return Tabla.desdeFilas(filas);
  }
```

> ☑ **BUENA PRÁCTICA —** Diseñar 'costuras' (seams) como estos constructores con nombre es una práctica profesional real: no se trata de contaminar el código de producción por las pruebas, sino de dar control explícito sobre lo que antes era aleatorio, cuando el control es necesario.

RONDA 10 — El motor de juego (integración completa)

🔴 ROJO — Escribe una prueba que falle

test/motor_juego_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:loteria_dolorena/logica/baraja.dart';
import 'package:loteria_dolorena/logica/tabla.dart';
import 'package:loteria_dolorena/logica/motor_juego.dart';
```

```
void main() {
  test('el motor detecta Loteria al completar una fila', () {
    final cartas = Baraja.crearBarajaCompleta().cartasRestantes; // orden fijo, sin mezclar
    final baraja = Baraja.desdeCartas(cartas);
    final tabla = Tabla.desdeFilas([
      cartas.sublist(0, 4),
      cartas.sublist(4, 8),
      cartas.sublist(8, 12),
      cartas.sublist(12, 16),
    ]);
    final motor = MotorJuego(baraja: baraja, tabla: tabla);

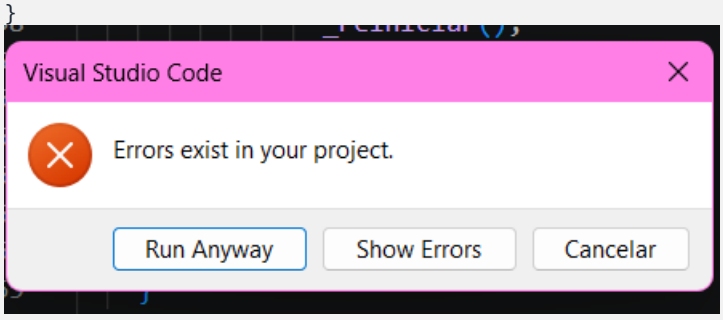
    // cantarSiguiente() usa removeLast(), asi que canta primero
    // las ultimas 4 cartas de la lista: exactamente la fila 3 de la tabla.
    for (var i = 0; i < 4; i++) {
      motor.cantarYMarcar();
    }

    expect(motor.haGanado, true);
  });

  test('el motor NO reporta Loteria si aun no se completa ningun patron', () {
    final cartas = Baraja.crearBarajaCompleta().cartasRestantes;
    final baraja = Baraja.desdeCartas(cartas);
    final tabla = Tabla.desdeFilas([
      cartas.sublist(0, 4),
      cartas.sublist(4, 8),
      cartas.sublist(8, 12),
      cartas.sublist(12, 16),
    ]);
    final motor = MotorJuego(baraja: baraja, tabla: tabla);

    motor.cantarYMarcar();
    motor.cantarYMarcar();

    expect(motor.haGanado, false);
  });
}
```



VERDE — El mínimo código para pasar

lib/logica/motor_juego.dart

```
import '../modelos/carta.dart';
import 'baraja.dart';
import 'tabla.dart';
import 'verificador_patrones.dart';

class MotorJuego {
  final Baraja baraja;
  final Tabla tabla;
```

```
  bool haGanado = false;

  MotorJuego({required this.baraja, required this.tabla});

  Carta? cantarYMarcar() {
    final carta = baraja.cantarSiguiente();
    if (carta == null) return null;

    tabla.marcar(carta);
    if (VerificadorPatrones.hayGanador(tabla.marcadas)) {
      haGanado = true;
    }
    return carta;
  }
}
```

Esta es la recompensa de las 9 rondas anteriores: MotorJuego no tiene que volver a implementar barajar, marcar, ni detectar patrones — simplemente orquesta piezas que ya probamos por separado, y las dos pruebas de integración lo confirman.

4. Ahora sí: la interfaz gráfica

Con las 10 rondas de lógica ya probadas, construimos la interfaz como una capa delgada que solo muestra el estado del MotorJuego y reacciona a un botón — tal como se espera en la cima de la pirámide de pruebas.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'logica/baraja.dart';
import 'logica/tabla.dart';
import 'logica/motor_juego.dart';

void main() => runApp(const LoteriaApp());

class LoteriaApp extends StatelessWidget {
  const LoteriaApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Loteria Dolorena',
      theme: ThemeData(primarySwatch: Colors.deepOrange, useMaterial3: true),
      home: const JuegoScreen(),
    );
  }
}

class JuegoScreen extends StatefulWidget {
  const JuegoScreen({super.key});
  @override
  State<JuegoScreen> createState() => _JuegoScreenState();
}

class _JuegoScreenState extends State<JuegoScreen> {
  late MotorJuego _motor;
  late Tabla _tabla;
  String _ultimaCarta = 'Toca Cantar para empezar';

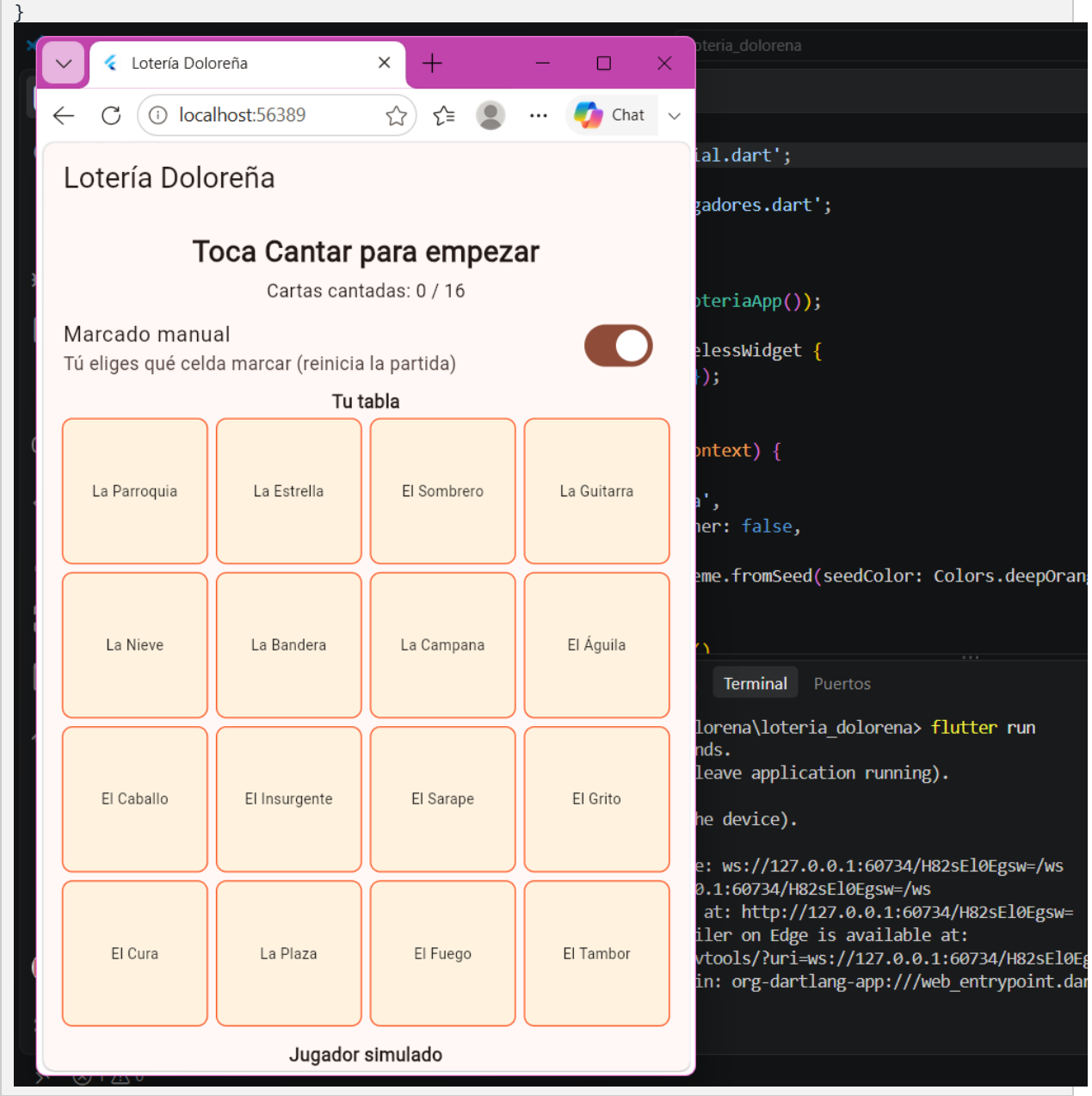
  @override
  void initState() {
    super.initState();
    _reiniciar();
  }

  void _reiniciar() {
    final baraja = Baraja.crearBarajaCompleta()..mezclar();
    _tabla = Tabla.generar(Baraja.crearBarajaCompleta().cartasRestantes);
    _motor = MotorJuego(baraja: baraja, tabla: _tabla);
    _ultimaCarta = 'Toca Cantar para empezar';
    setState(() {});
  }

  void _cantar() {
    final carta = _motor.cantarYMarcar();
    setState(() => _ultimaCarta = carta?.nombre ?? 'Ya no quedan cartas');
    if (_motor.haGanado) {
      showDialog(
        context: context,
        builder: (_) => AlertDialog(
          title: const Text('Loteria!'),
          content: const Text('Completaste una linea, columna o diagonal.'),
          actions: [
            TextButton(
              onPressed: () {
```

```
                Navigator.pop(context);
                _reiniciar();
              },
              child: const Text('Jugar de nuevo'),
            ),
          ],
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Loteria Dolorena')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(16),
            child: Text(_ultimaCarta, style: const TextStyle(fontSize: 22, fontWeight:
FontWeight.bold)),
          ),
          Expanded(
            child: GridView.builder(
              padding: const EdgeInsets.all(12),
              gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(crossAxisCount: 4),
              itemCount: 16,
              itemBuilder: (context, i) {
                final f = i ~/ 4;
                final col = i % 4;
                final carta = _tabla.filas[f][col];
                final marcada = _tabla.marcadas[f][col];
                return Container(
                  margin: const EdgeInsets.all(4),
                  decoration: BoxDecoration(
                    color: marcada ? Colors.deepOrange : Colors.orange.shade50,
                    border: Border.all(color: Colors.deepOrange),
                    borderRadius: BorderRadius.circular(8),
                  ),
                  child: Center(
                    child: Text(
                      carta.nombre,
                      textAlign: TextAlign.center,
                      style: TextStyle(fontSize: 11, color: marcada ? Colors.white :
Colors.black87),
                    ),
                  ),
                );
              },
            ),
          ),
          Padding(
            padding: const EdgeInsets.all(16),
            child: ElevatedButton.icon(
              onPressed: _cantar,
              icon: const Icon(Icons.campaign),
              label: const Text('Cantar siguiente carta'),
            ),
          ),
        ],
      ),
    );
  }
```



5. Cómo ejecutar todo

1. Ejecuta flutter test para confirmar que las 10 rondas de lógica siguen en verde (deberían ser 11 pruebas en total).
2. Ejecuta flutter run para jugar.
3. Presiona 'Cantar siguiente carta' repetidamente hasta que aparezca el diálogo de '¡Lotería!'.

6. Checklist de este proyecto TDD

✓	Verificación
☐	Cada clase de lib/logica/ y lib/modelos/ tiene al menos una prueba escrita ANTES que su código
☐	Ninguna prueba de lógica (baraja, tabla, patrones, motor) depende de widgets ni de un emulador

☐	Los constructores desdeCartas y desdeFilas existen únicamente para dar control explícito en las pruebas
☐	MotorJuego no reimplementa barajar, marcar ni detectar patrones — solo orquesta
☐	flutter test pasa sin advertencias antes de ejecutar flutter run

7. Retos de extensión

> 🎯 **RETO —** Agrega la detección de 'tabla llena' (las 16 celdas marcadas) como un premio mayor distinto a completar solo una línea — con su propia prueba antes que su implementación.

- Agrega un contador de cuántas cartas se han cantado, mostrado en pantalla.
- Permite que el jugador elija manualmente qué celda marcar (en lugar de marcar automático), y prueba que un marcado incorrecto no cuenta como válido.
- Agrega una segunda tabla (un segundo jugador simulado) y detecta quién gana primero.

⏳ **INSIGNIA DESBLOQUEADA: Cinturón Negro de Proyecto — construiste un juego completo de principio a fin, guiado 100% por pruebas**